

# How Frontier AI Models Treat Information Sources in AI-for-Software-Engineering Questions

AJOY DEY, Bangladesh University of Engineering and Technology, Bangladesh

AI-assisted software engineering is now discussed through many kinds of sources: software engineering conferences, programming-language venues, arXiv papers, benchmarks, company blogs, documentation, GitHub repositories, and practitioner reports. This project studies how frontier and open-weight AI models treat those sources when asked questions about AI in software engineering. I designed a reproducible prompt-based study with a 30-venue conference baseline, 10 prompts, 30 raw model responses, parsers for rankings and citations, and analysis scripts for ranking agreement, source-type distribution, domain concentration, and link validation. The collected responses came from OpenRouter free routes for Gemma 4 31B, GPT-OSS 120B, and Nemotron 3 Ultra. The pipeline extracted 205 ranking rows, 86 citation rows, and 86 source links. Results show that models usually rank ICSE, FSE, ASE, ISSTA, and MSR highly for AI-for-software-engineering, but source-heavy prompts draw heavily from arXiv and web-accessible resources. The study also finds practical limitations: free API routes were rate-limited, source links include dead or unverifiable URLs, and model rankings depend strongly on prompt depth and source-pressure instructions. The repository contains code, data, raw responses, parsed tables, figures, and this report so the study can be reproduced and extended.

**Date:** 2 July 2026

## 1 Introduction

Modern software developers often solve problems by moving between academic and industrial sources. In my daily software work with AI, I regularly search academic papers for durable research ideas and industrial resources for practical implementation details, APIs, benchmarks, and deployment guidance. This made the project personally meaningful: if AI assistants are becoming part of software engineering work, then it is important to understand whether they value established research sources, whether they overuse recent web material, and whether they cite exact sources reliably.

This report responds to the project request to explore idiosyncrasies across frontier models in the treatment of information sources for AI in software engineering. The project has two main goals:

- (1) study model understanding of source impact by asking models to rank conferences and venues related to AI-assisted software engineering; and
- (2) study retrieval and attribution behavior by asking models for exact links to papers, blogs, repositories, documentation, benchmarks, and other internet sources.

The final artifact is not only this report. It is a reproducible research package containing prompts, raw responses, parsing scripts, analysis scripts, figures, tables, and limitations.

## 2 Research Questions

The study was organized around seven research questions:

- RQ1:** Which conferences do AI models consider most important for AI in software engineering?
- RQ2:** Do model-generated conference rankings correlate with a baseline derived from CORE-style ranks and CSRankings areas?
- RQ3:** Do models prefer academic papers, company blogs, documentation, arXiv papers, GitHub repositories, news articles, or general web pages?

**RQ4:** Do models cite sources from their own organization or ecosystem more often than competing sources?

**RQ5:** Do models over-prioritize recent LLM and code-generation work compared with older software engineering research?

**RQ6:** Do shallow prompts produce more web/blog sources while deeper prompts produce more academic citations?

**RQ7:** Which responses contain dead, unverifiable, ambiguous, or likely hallucinated source metadata?

### 3 Dataset Design

#### 3.1 Conference Baseline

The first dataset is a 30-venue master conference list stored in `data/conferences/master_conference_list`. The list covers:

- software engineering venues: ICSE, FSE, ASE, ISSTA, MSR, ICSME, SANER, RE, MODELS, ICPC, ICST, COMPSAC, and SAC;
- programming-language and formal-methods venues: PLDI, OOPSLA, POPL, ECOOP, CAV, TACAS, FM, and VMCAI; and
- adjacent AI, data, web, and HCI venues: NeurIPS, ICML, ICLR, AAAI, IJCAI, KDD, WWW, CHI, and UIST.

The columns are conference, acronym, full\_name, field, core\_rank, csrankings\_area, and baseline\_rank. I intentionally included lower-rank or more specialized venues such as COMPSAC, ICST, and SAC. This tests whether models only repeat the most prestigious venues or can also recognize specialized sources that may be useful for practical AI-for-SE questions.

CORE and CSRankings are used as comparison signals, not as absolute truth. CORE focuses on venue ranking categories, while CSRankings is area-based and publication-count oriented [1, 3]. Therefore, the baseline rank is a pragmatic comparison instrument rather than an objective measure of research quality.

#### 3.2 Prompt Battery

The prompt set contains 10 prompts stored in `data/prompts/prompt_set_v1.md`, with prompt metadata in `data/prompts/prompt_metadata.csv`. The prompts vary by depth and source pressure:

- conference ranking prompts;
- sourced conference ranking prompts;
- influential-paper prompts;
- exact-source disclosure prompts;
- recent-work prompts;
- area-mapping prompts;
- practitioner-guidance prompts;
- hallucination-stress prompts;
- lower-rank venue prompts; and
- source-validation audit prompts.

Most prompts request JSON with ranked\_items, references, and uncertainty\_notes. This structure makes the responses easier to parse while still allowing models to express uncertainty.

4 Model Collection Method

4.1 Models

The final collection used OpenRouter free routes for three models [5]:

- google/gemma-4-31b-it:free
- openai/gpt-oss-120b:free
- nvidia/nemotron-3-ultra-550b-a55b:free

The collection script is `src/query_models.py`. It saves every raw response as JSON under `data/raw_responses/`. Each file includes the provider, model, prompt id, run id, timestamp, temperature, system prompt, user prompt, API parameters, and exact response text.

4.2 Collection Parameters

The target decoding setting was temperature 0. Because the free model routes were rate-limited, some calls used smaller `max_tokens` limits to avoid timeouts. The final raw dataset contains 30 real API response files. The composition is shown in Table 1.

Table 1. Raw response collection summary.

| Model and run             | Provider   | Responses |
|---------------------------|------------|-----------|
| Gemma 4 31B, final01      | OpenRouter | 7         |
| GPT-OSS 120B, final01     | OpenRouter | 2         |
| Nemotron 3 Ultra, final01 | OpenRouter | 10        |
| Nemotron 3 Ultra, final02 | OpenRouter | 10        |
| Nemotron 3 Ultra, final03 | OpenRouter | 1         |
| Total                     |            | 30        |

The study originally aimed for a balanced 10 prompts by 3 models matrix. OpenRouter free-route rate limits prevented a perfectly balanced collection: Gemma and GPT-OSS were intermittently rate-limited, while Nemotron was slower but more reliable. Instead of fabricating or filling missing data, I collected repeated Nemotron runs. This produces a real 30-response dataset and also allows a small stability check across repeated runs.

5 Parsing and Analysis Pipeline

The pipeline is designed to be reproducible from the repository root.

```
python src/run_pipeline.py
python src/validate_links.py \
  --input data/parsed/source_links.csv \
  --output data/parsed/source_links_validated.csv
```

5.1 Parsing

`src/parse_responses.py` extracts three tables:

- conference rankings: `data/parsed/conference_rankings.csv`;
- citations: `data/parsed/citations.csv`; and
- source links: `data/parsed/source_links.csv`.

The parser first attempts JSON parsing. If the response is malformed or wrapped in markdown fences, it uses a lenient JSON extraction fallback. It also extracts URLs, DOIs, and arXiv identifiers with regular expressions when a model does not provide clean structured output.

## 5.2 Ranking Metrics

`src/analyze_rankings.py` joins model-ranked venues against the conference baseline. It computes:

- Spearman correlation with baseline rank;
- Kendall tau with baseline rank;
- Spearman correlation with numeric CORE score;
- top-5 overlap with the baseline; and
- mean absolute rank difference.

## 5.3 Source Metrics

`src/analyze_sources.py` measures source-type distribution, domain concentration, citation overlap using Jaccard similarity, publication-year distribution, and hallucinated-citation candidates. Source types include `academic_paper`, `conference_page`, `arxiv`, `company_blog`, `documentation`, `github_repository`, `news`, `benchmark_or_dataset`, and `unknown`.

`src/validate_links.py` performs automated URL validation. A link is marked `valid`, `dead_link`, `unverifiable`, or `ambiguous`. This is not a perfect hallucination detector, but it is a useful first pass for identifying suspicious source claims.

# 6 Results

## 6.1 Parsed Dataset Size

The final pipeline run extracted:

- 205 ranking rows;
- 86 citation rows;
- 86 source-link rows; and
- 86 validated source-link rows.

These counts show that the prompt design was able to elicit both ranked venue judgments and exact source disclosures.

## 6.2 RQ1 and RQ2: Conference Ranking Behavior

The ranking prompts generally placed ICSE, FSE, ASE, ISSTA, and MSR near the top. This is reasonable because these venues are central to software engineering, automated software engineering, software testing, and mining software repositories. AI-specific venues such as ICLR, NeurIPS, and ICML also appeared, especially when models connected AI-for-SE to LLM and code-model research.

Table 2 reports selected ranking metrics. The strongest agreement came from GPT-OSS on the sourced ranking prompt and Nemotron on the lower-rank venue prompt. The lower-rank prompt improved coverage of specialized software engineering venues and produced a top-5 overlap of 1.00 for Nemotron final02.

## 6.3 RQ3: Source Type Preference

Table 3 shows the aggregate source-type distribution. arXiv dominates the extracted links with 48 out of 86 source links. This suggests that when asked for exact sources, the models often prefer accessible preprint links over ACM/IEEE pages or older conference/journal records. GitHub repositories and conference pages appear, but less frequently. Nineteen links were classified as unknown, usually because they pointed to ranking sites, general project pages, or domains not covered by the first-version classifier.

Table 2. Selected ranking agreement metrics. Higher Spearman, Kendall, and top-5 overlap indicate stronger agreement with the baseline. Lower mean absolute rank difference is better.

| Model            | Prompt               | Matched | Spearman | Kendall | Top-5 | Mean Abs. Diff. |
|------------------|----------------------|---------|----------|---------|-------|-----------------|
| Gemma 4 31B      | P01 ranking          | 8       | 0.643    | 0.571   | 0.60  | 4.25            |
| Gemma 4 31B      | P02 sourced ranking  | 11      | 0.536    | 0.455   | 0.60  | 5.73            |
| Nemotron 3 Ultra | P01 ranking          | 9       | 0.700    | 0.611   | 0.60  | 3.11            |
| Nemotron 3 Ultra | P02 sourced ranking  | 15      | 0.682    | 0.581   | 0.80  | 6.00            |
| Nemotron 3 Ultra | P09 lower-rank check | 19      | 0.788    | 0.708   | 1.00  | 5.53            |
| GPT-OSS 120B     | P02 sourced ranking  | 9       | 1.000    | 1.000   | 1.00  | 2.67            |

Table 3. Aggregate source-type distribution.

| Source type       | Count |
|-------------------|-------|
| arXiv             | 48    |
| unknown           | 19    |
| conference page   | 6     |
| GitHub repository | 6     |
| academic paper    | 4     |
| personal blog     | 2     |
| company blog      | 1     |

6.4 RQ4: Domain Concentration

The most common domain was arxiv.org. For Gemma, the top extracted domains included arxiv.org, github.com, csrankings.org, portal.core.edu.au, owasp.org, and huggingface.co. For Nemotron, the strongest concentration was also arxiv.org, followed by domains such as conf.researchr.org, docs.github.com, doi.org, github.blog, and github.com. This pattern suggests that models used broadly available academic and developer-facing web sources more than closed publisher pages.

The evidence for self-organization preference is limited in this dataset. There are some GitHub and GitHub documentation links, but the collected models did not overwhelmingly cite their own provider ecosystems. A larger balanced dataset would be needed to make a strong claim about organizational bias.

6.5 RQ5 and RQ6: Recency and Prompt Sensitivity

Recent LLM and code-generation work appeared frequently. Examples include Codex-style code generation papers [2], SWE-bench style software engineering agent benchmarks [4], and other arXiv-accessible work. This indicates a recency pull toward LLM-era software engineering research. However, older and established software engineering venues still appeared in ranking prompts, especially ICSE, FSE, ASE, ISSTA, and MSR.

Prompt sensitivity was visible. Ranking prompts emphasized conferences and prestige. Exact-source prompts produced many arXiv URLs, GitHub links, and web-accessible pages. The lower-rank venue prompt changed the answer surface: it encouraged inclusion of specialized venues that may otherwise be omitted. This supports the idea that shallow prompts can reinforce common top-venue

answers, while deeper prompts with explicit source pressure expose richer and more diverse source choices.

## 6.6 RQ7: Link Validity and Hallucination Risk

Automated validation of 86 extracted links produced:

- 72 valid links;
- 12 dead links; and
- 2 unverifiable links.

Dead and unverifiable links are important because they may indicate hallucinated URLs, changed URLs, temporary access failures, or websites that block automated validation. I therefore treat validation as a warning signal rather than final proof of hallucination. Manual checking is still needed for high-stakes claims.

## 7 Discussion

### 7.1 What I Learned

This project showed me that AI assistants do not simply retrieve neutral source lists. Their answers are shaped by prompt wording, source pressure, API behavior, and the model's learned associations between a topic and well-known venues. When asked broadly, models tend to name obvious high-impact venues. When asked for exact sources, they often move toward accessible links such as arXiv, GitHub, documentation, and conference pages.

This matters for software engineering practice. In real software work, an engineer often needs both academic depth and industrial immediacy. Academic papers help with foundations and evidence; documentation, repositories, and company blogs help with implementation. A good AI assistant should help users move across both worlds, while clearly separating verified sources from uncertain or generated metadata.

### 7.2 Threats to Validity

**API availability.** The strongest threat is the OpenRouter free-route rate limit. Gemma and GPT-OSS could not be collected for all 10 prompts during the experiment window. I did not fabricate missing responses. Instead, I documented the limitation and collected repeated Nemotron runs.

**Baseline validity.** CORE and CSRankings are useful signals but they measure different things. A venue can be specialized and useful even if it is not ranked at the top of a general prestige baseline.

**Parsing validity.** The parser handles structured JSON and fallback text extraction, but model responses are messy. Some venue names or citations may be missed or normalized imperfectly.

**Link validation.** Automated link validation can fail because of temporary network problems, redirects, bot blocking, publisher restrictions, or changed URLs. Manual checking remains necessary.

**Model versioning.** Free API routes may change underlying models or providers over time. Raw responses and timestamps are stored to preserve the actual observed data.

## 8 Reproducibility Package

The repository includes:

- README.md with setup and run instructions;
- requirements.txt for dependencies;
- data/conferences/ with baseline venue data;
- data/prompts/ with prompts and metadata;
- data/raw\_responses/ with 30 raw API responses;
- data/parsed/ with parsed rankings, citations, and links;

- src/ with collection, parsing, analysis, visualization, and validation scripts;
- outputs/tables/ with result tables; and
- outputs/figures/ with generated figures.

The main reproduction command is:

```
python src/run_pipeline.py
```

## 9 Conclusion

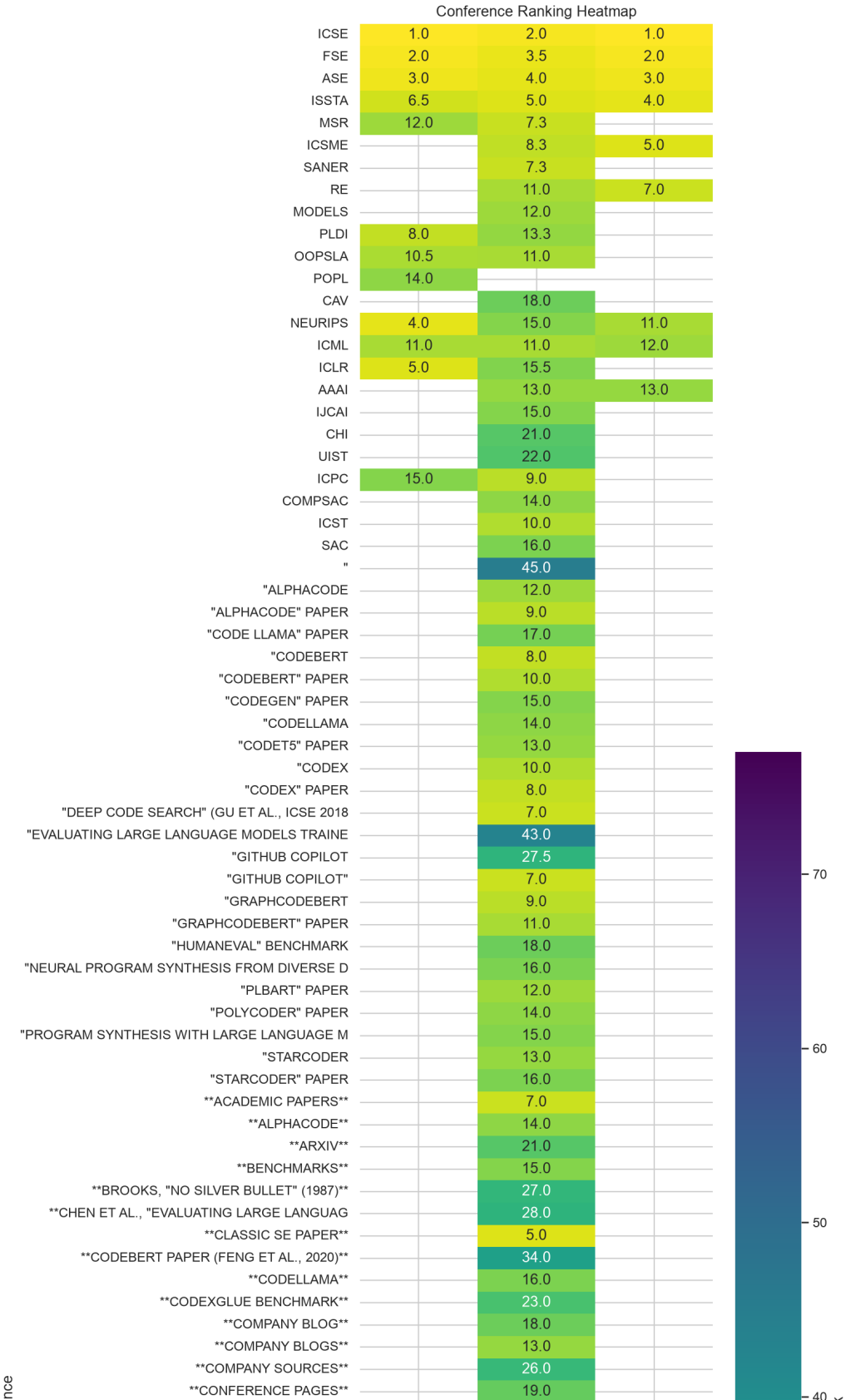
This project designed and executed a reproducible study of how AI models treat information sources in AI-for-software-engineering questions. The results show that models recognize the centrality of major software engineering venues such as ICSE, FSE, ASE, ISSTA, and MSR, but source-heavy prompts shift responses toward arXiv and other easily linkable web sources. The lower-rank venue prompt was useful because it encouraged models to include specialized venues that matter for practical software engineering research but may be overlooked in shallow prompts.

The project also exposed practical research challenges. Free API routes can be rate-limited, model output formats can be inconsistent, and exact URLs can be dead or unverifiable. These are not just inconveniences; they are part of the phenomenon being studied. If AI systems are used as research assistants or software engineering assistants, their source behavior must be measured, validated, and documented.

Overall, this study provides a working first version of a reproducible benchmark for source treatment in AI-for-software-engineering answers. Future work should collect balanced runs across more models, add Crossref or OpenAlex metadata validation, and compare API responses with web-interface responses from chat tools used by everyday software engineers.

## References

- [1] Emery D. Berger. 2026. CSRankings: Computer Science Rankings. <https://csrankings.org/> Accessed July 2026.
- [2] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [3] Computing Research and Education Association of Australasia. 2026. CORE Conference Ranking Portal. <http://portal.core.edu.au/conf-ranks/> Accessed July 2026.
- [4] Carlos E. Jimenez et al. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 <https://arxiv.org/abs/2310.06770>
- [5] OpenRouter. 2026. OpenRouter Model API. <https://openrouter.ai/> Used for free-route model collection in July 2026.





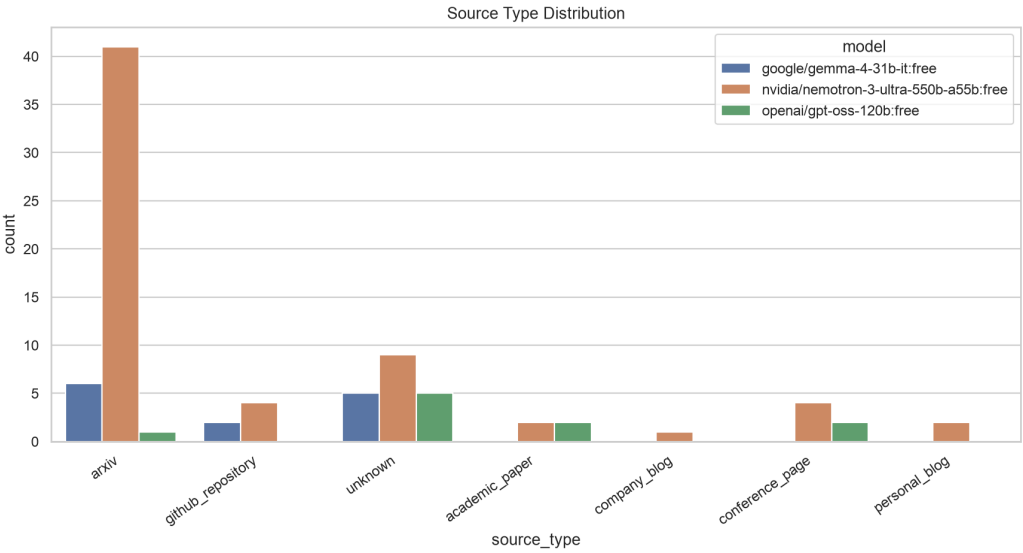


Fig. 2. Source type distribution by model and prompt.

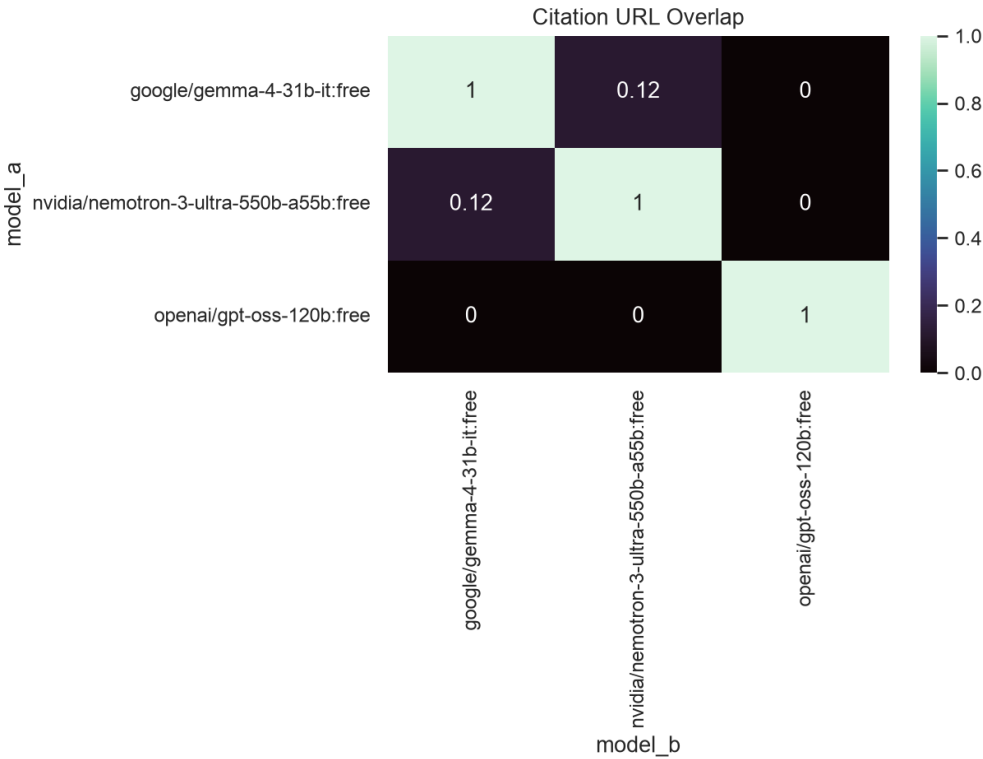


Fig. 3. Citation URL overlap using Jaccard similarity.

Publication Year Distribution

No data yet

Fig. 4. Publication year distribution extracted from citation metadata.